

THE GEORGE
WASHINGTON
UNIVERSITY

WASHINGTON, DC

DATA SCIENCE PROGRAM

CAPSTONE REPORT - FALL 2025

Adaptive Bandit Algorithms for Drifting Users

Hema Chandra Puchakayala

Supervised by
Tyler Wallett Amir Jafari

Abstract

Modern recommender systems operate in environments where user preferences drift over time, breaking the stationarity assumptions of classical bandit algorithms. This project builds a synthetic drifting-user simulator and evaluates four adaptive multi-armed bandit methods—Non-Stationary ϵ -Greedy, Sliding-Window UCB, CUSUM-UCB, and EXP3.S—on their ability to track changing reward distributions. Each algorithm is run for 10,000 interaction steps across 50 random seeds, and performance is measured using rolling average and cumulative reward with confidence intervals. The results show that Sliding-Window UCB achieves the highest long-run reward, with CUSUM-UCB and Non-Stationary ϵ -Greedy forming a competitive middle tier, while the adversarial EXP3.S underperforms in this stochastic drift setting. These findings suggest that methods which explicitly discount or reset outdated observations are better suited to gradual user drift than adversarial bandit algorithms tuned for worst-case regret.

Contents

1	Introduction	3
2	Problem Statement	3
3	Related Work	3
4	Solution and Methodology	4
4.1	Drifting-User Environment	4
4.2	Bandit Algorithms	4
4.3	Experimental Protocol	7
5	Results and Discussion	8
5.1	Experimentation Protocol	8
5.2	Graphs	8
6	Discussion	11
7	Conclusion	11

1 Introduction

Online platforms rely on recommender systems to personalize content, ads, and products to individual users. A central challenge in these systems is *user drift*: preferences evolve over time due to changes in taste, context, and external trends. When rewards change independently of the system’s actions, the problem is more naturally modeled as a non-stationary multi-armed bandit (MAB) rather than a full Markov decision process.

This project studies how different adaptive bandit algorithms respond to user drift in a controlled simulation environment. Users are represented by latent preference vectors over a small set of content categories, and their preferences follow a drifting process that gradually favors one document over another. Within this environment, we compare four bandit strategies that have been proposed for non-stationary or adversarial settings and quantify how well they track the shifting reward landscape.

2 Problem Statement

The goal of this project is to evaluate and compare adaptive bandit algorithms for a recommendation scenario where user preferences drift over time. We focus on a simplified setting with two documents (arms) and a single user whose latent preferences evolve independently of the recommender’s actions.

The main challenges are:

- Designing a synthetic yet realistic drifting-user environment that is reproducible across experiments.
- Selecting and correctly implementing bandit algorithms that embody different adaptation mechanisms (sliding windows, change detection, constant step-size, adversarial weighting).
- Defining metrics and an experimental protocol that allow a fair comparison of models over long horizons and multiple random seeds.

Our objective is not to build a production-ready recommender, but to isolate and study the adaptation behavior of these algorithms under controlled drift.

3 Related Work

Non-stationary bandits have been studied through a variety of approaches, including discounted UCB, sliding-window estimators, change-detection-based methods, and adversarial algorithms such as EXP3.[1, 2, 3, 4] Sliding-Window UCB (SW-UCB) restricts its statistics to the most recent τ observations, allowing it to forget outdated rewards quickly. Non-Stationary ϵ -Greedy replaces sample averages with exponential recency-weighted estimates, providing a simple baseline for drift.

CUSUM-UCB augments the classical UCB index with a two-sided CUSUM detector that restarts an arm when a significant mean shift is detected, making it well-suited to piecewise-stationary environments. EXP3 and its variants, such as EXP3.S, treat rewards as an arbitrary sequence and maintain exponential weights over arms, mixing exploitation with uniform exploration to obtain adversarial regret guarantees.

Our work differs from previous studies by placing these four algorithms in a common drifting-user simulator tailored to recommender-style scenarios. The simulator exposes a unified interface, making it straightforward to add new algorithms and regenerate comparable results under alternative drift regimes.

4 Solution and Methodology

4.1 Drifting-User Environment

We implement a function `create_environment` that generates a *drifting* bandit environment with K arms and T timesteps. At $t = 0$ the user’s preference vector over arms is uniform. At each subsequent step the vector is perturbed by Gaussian noise and renormalized to form a valid probability distribution. This distribution is interpreted as the probability of a click or reward for each arm, from which Bernoulli rewards are sampled.

For the experiments in this report we set $K = 2$ documents and $T = 10,000$ interaction steps. Each run is initialized with a different random seed so that user trajectories and rewards differ across experiments while sharing the same drift dynamics.

4.2 Bandit Algorithms

A wide range of strategies has been developed to address non-stationarity in multi-armed bandit problems, particularly in settings where the reward distribution drifts or changes abruptly over time. Classical bandit algorithms such as UCB and ε -greedy assume stationary reward distributions and therefore struggle when user preferences evolve, as their estimates place equal weight on outdated and recent observations. To mitigate this limitation, several adaptive variants have been proposed in the literature, each incorporating mechanisms to discount stale information, detect distributional shifts, or explicitly reset statistics after suspected changes.

This section reviews four representative approaches that form the basis of our comparative study: Sliding-Window UCB (SW-UCB), Non-Stationary ε -Greedy, EXP3.S, and CUSUM-UCB. These methods span both stochastic and adversarial formulations of non-stationary bandits and provide a spectrum of adaptation mechanisms, including sliding-window averaging, exponential recency-weighted updates, weight-sharing regularization, and online change detection. By examining these algorithms, we establish a comprehensive foundation for evaluating how different adaptation strategies respond to user drift in recommendation-style environments.

- **Sliding-Window UCB (SW-UCB)**[1]: SW-UCB is a variant of UCB designed for non-stationary environments with abrupt changes. Instead of using all past observations, it computes estimates using only the most recent τ rounds, allowing rapid adaptation to changing reward distributions.

The sliding-window empirical mean for arm i at time t is

$$\bar{X}_t(\tau, i) = \frac{1}{N_t(\tau, i)} \sum_{s=t-\tau+1}^t X_s(i) 1_{\{I_s=i\}},$$

where

$$N_t(\gamma, i) = \sum_{s=1}^t \gamma^{t-s} 1_{\{I_s=i\}},$$

The is the number of times the arm i was selected in the last τ rounds, $\gamma \in (0, 1)$ is the discount factor controlling how quickly past observations are forgotten and ξ is a some appropriate constant.

SW-UCB constructs an upper-confidence bound using:

$$c_t(\tau, i) = B \sqrt{\frac{\xi \log(t \wedge \tau)}{N_t(\tau, i)}},$$

where B is the upper-bound on the rewards, $t \wedge \tau$ denotes the minimum of t and τ and $\xi > 1/2$ controls exploration.

Action selection is performed by

for $t = 1$ to K , play arm $I_t = t$;

for $t = K + 1$ to T , play arm

$$I_t = \arg \max_{1 \leq i \leq K} (\bar{X}_t(\tau, i) + c_t(\tau, i)).$$

After receiving the reward $X_t(a_t)$, the sliding-window statistics are updated by discarding observations older than τ rounds and inserting the new one:

$$\begin{aligned} N_{t+1}(\gamma, i) &= \sum_{s=t-\tau+2}^{t+1} \gamma^{t+1-s} 1_{\{I_s=i\}}, \\ \bar{X}_{t+1}(\tau, a) &= \frac{1}{N_{t+1}(\tau, a)} \sum_{s=t-\tau+2}^{t+1} X_s(i) 1_{\{I_s=i\}}, \\ c_{t+1}(\tau, i) &= B \sqrt{\frac{\xi \log((t+1) \wedge \tau)}{N_{t+1}(\tau, i)}}. \end{aligned}$$

- **Non-Stationary Epsilon-Greedy**[2]: This is an adaptation of the classical ε -greedy method for environments in which the reward distributions drift over time. Instead of sample averages, the action-value estimates use a *constant step-size* update, which implements an exponential recency-weighted average that discounts older rewards.

The estimate of the value of arm i at time t is updated according to the incremental constant-step-size rule

$$Q_{t+1}(i) = Q_t(i) + \alpha (R_t - Q_t(i)) 1_{\{I_t=i\}},$$

where $\alpha \in (0, 1]$ is the constant step-size parameter. Unrolling this recursion gives the exponential recency-weighted form

$$Q_{t+1}(i) = (1 - \alpha)^t Q_1(i) + \sum_{s=1}^t \alpha (1 - \alpha)^{t-s} R_s 1_{\{I_s=i\}},$$

showing that recent rewards receive more weight than older ones, with decay factor $(1 - \alpha)$.

Action selection is performed using the ε -greedy policy:

for $t = 1$ to K , play arm $I_t = t$;

for $t = K + 1$ to T , select arm

$$I_t = \begin{cases} \text{a random arm in } \{1, \dots, K\}, & \text{with probability } \varepsilon, \\ \arg \max_{1 \leq i \leq K} Q_t(i), & \text{with probability } 1 - \varepsilon. \end{cases}$$

After observing reward R_t , the value estimates are updated using the constant step-size rule:

$$Q_{t+1}(I_t) = Q_t(I_t) + \alpha (R_t - Q_t(I_t)).$$

Because the estimate $Q_t(i)$ places exponentially decaying weight on past rewards with weights proportional to $(1 - \alpha)^k$ for k -step-old observations method adapts effectively to non-stationary reward distributions. Larger values of α allow faster adaptation, while smaller values provide greater stability.

- **EXP3.S**[4]: EXP3.S is a variant of the EXP3 algorithm proposed in Auer et al. (1995) [5] designed to control regret against *arbitrary* (possibly switching) action sequences. Rather than competing only with the best single arm, EXP3.S adapts to piecewise-stationary environments by incorporating an additional stabilization term in the weight update that helps the algorithm track changing optimal arms.

Like EXP3, it maintains exponential weights over the K arms and mixes between exploitation and uniform exploration. However, EXP3.S adds a regularization term proportional to α that penalizes rapid fluctuations and allows regret bounds scaling with the *hardness* of the comparator sequence (i.e., the number of switches throughout the sequence).

Action selection:

At each round t , compute the probability distribution

$$p_i(t) = (1 - \gamma) \frac{w_i(t)}{\sum_{j=1}^K w_j(t)} + \frac{\gamma}{K}, \quad i = 1, \dots, K.$$

Then draw action I_t according to the distribution $p(t)$.

Reward estimation:

After receiving reward $x_{I_t}(t) \in [0, 1]$, define the unbiased importance-weighted estimator

$$\hat{x}_j(t) = \begin{cases} \frac{x_j(t)}{p_j(t)}, & j = I_t, \\ 0, & j \neq I_t. \end{cases}$$

Weight update:

For each arm $j = 1, \dots, K$, update

$$w_j(t+1) = w_j(t) \exp\left(\frac{\gamma}{K} \hat{x}_j(t)\right) + \frac{e\alpha}{K} \sum_{i=1}^K w_i(t).$$

The additive term

$$\frac{e\alpha}{K} \sum_{i=1}^K w_i(t)$$

helps the algorithm adapt to environments where the best arm switches, enabling regret bounds that depend on the number of switches rather than on T alone.

- **CUSUM-UCB**[3]: CUSUM-UCB is a change-detection based bandit algorithm designed for piecewise-stationary environments. It augments the classical UCB index with a two-sided online CUSUM detector that monitors each arm for statistically significant shifts in its reward distribution. When a change is detected on arm i , the algorithm *restarts* that arm by resetting its empirical mean, sample count, and the associated CUSUM statistics. This prevents outdated pre-change rewards from contaminating the UCB estimate and enables rapid recovery after abrupt changes. For each arm, the first M observations are used to estimate the pre-change mean,

$$\hat{u}_0 = \frac{1}{M} \sum_{k=1}^M y_k,$$

and for each subsequent sample y_k , the CUSUM updates use the steps

$$\begin{aligned} (s_k^+, s_k^-) &= (y_k - \hat{u}_0 - \varepsilon, \hat{u}_0 - y_k - \varepsilon) \mathbf{1}_{\{k > M\}}, \\ g_k^+ &= \max\{0, g_{k-1}^+ + s_k^+\}, \quad g_k^- = \max\{0, g_{k-1}^- + s_k^-\}. \end{aligned}$$

A change is detected whenever either statistic crosses the threshold h , i.e.,

$$g_k^+ \geq h \quad \text{or} \quad g_k^- \geq h,$$

upon which the corresponding arm is restarted: its restart time τ_i is set to the current round, and the CUSUM variables (g_k^+, g_k^-) are reset to zero.

The UCB part of the algorithm uses only post-restart statistics. Let τ_i denote the last restart time of arm i . Then its valid sample count, empirical mean, and confidence padding at time t are

$$N_t(i) = \sum_{s=\tau_i}^t 1_{\{I_s=i\}}, \quad \bar{X}_t(i) = \frac{1}{N_t(i)} \sum_{s=\tau_i}^t X_s(i) 1_{\{I_s=i\}},$$

$$C_t(i) = \sqrt{\frac{\xi \log n_t}{N_t(i)}}, \quad n_t = \sum_{j=1}^K N_t(j),$$

where $\xi > 0$ is the UCB exploration constant.

Action selection combines UCB exploitation with uniform sampling at rate α , as required to feed the CUSUM detectors:

$$I_t = \begin{cases} \arg \max_{1 \leq i \leq K} (\bar{X}_t(i) + C_t(i)), & \text{with probability } 1 - \alpha, \\ \text{a uniformly random arm in } \{1, \dots, K\}, & \text{with probability } \alpha/K. \end{cases}$$

Update rule: After observing reward $X_t(I_t)$, the algorithm

1. updates the CUSUM statistics $g_k^+, g_k^-, s_k^+, s_k^-$ for the played arm;
2. if a CUSUM alarm is raised, resets $\tau_{I_t} = t + 1$ and clears the CUSUM state;
3. otherwise, updates $(N_t(i), \bar{X}_t(i), C_t(i))$ for the played arm using the post-restart definitions above.

Because all pre-change samples of an arm are discarded upon detection, CUSUM-UCB avoids the slow recovery faced by classical UCB when reward means abruptly decrease.

All algorithms are implemented in a common interface with `train` methods that accept a matrix of rewards and return per-step reward sequences and arm selections for analysis.

4.3 Experimental Protocol

For each algorithm we run 50 independent experiments with distinct seeds. In every run, the environment first generates a full $T \times K$ reward matrix from the drifting-user process. The algorithm then interacts with this matrix for T steps, observing one reward per round and updating its internal state.

We log:

- The realized reward R_t at each step.
- A one-hot encoded action vector indicating the selected arm.

For each model and metric we then average over the 50 runs and compute pointwise 95% confidence intervals based on the empirical standard error.

5 Results and Discussion

5.1 Experimentation Protocol

The main evaluation metrics are:

- *Rolling average reward*

$$\bar{R}(t) = \frac{1}{t} \sum_{s=1}^t R_s,$$

which reflects adaptation speed and stability over time.

- *Cumulative reward*

$$R_{\text{cum}}(T) = \sum_{t=1}^T R_t,$$

which captures total utility over the horizon.

Both metrics are reported as mean curves with 95% confidence bands over 50 runs. The best model is identified as the algorithm with the highest final cumulative reward $R_{\text{cum}}(T)$, while overlapping confidence intervals are used to judge whether differences are practically meaningful.

5.2 Graphs

Figure 1 illustrates the average and cumulative reward of each arm in the drifting environment, aggregated over all runs. Initially both arms are symmetric, but as the user’s latent preferences drift, one arm becomes consistently more rewarding. The widening gap between curves shows that the environment poses a genuine tracking challenge for bandit algorithms.

Figure 2 compares the four algorithms. The top panel shows the rolling average reward; the bottom panel shows cumulative reward. All stochastic bandits improve quickly from the initial 0.5 baseline, but Sliding-Window UCB rises fastest and stabilizes at the highest average reward. CUSUM-UCB and Non-Stationary ε -Greedy plateau at slightly lower levels, whereas EXP3.S increases slowly and remains substantially below the others throughout the horizon.

From these graphs we observe that Sliding-Window UCB is the clear winner in terms of both adaptation speed and total reward. Its finite window allows it to forget obsolete rewards and focus on the most recent regime. CUSUM-UCB performs similarly but with occasional dips corresponding to detected changes and subsequent restarts. The Non-Stationary ε -Greedy baseline eventually tracks the better arm but does so more slowly because its constant step-size must trade off reactivity against variance.

EXP3.S, designed for adversarial bandits, behaves conservatively in this stochastic drift scenario. By updating only the weight of the played arm using an importance-weighted estimate and mixing with uniform exploration, EXP3.S obtains strong worst-case guarantees but fails to fully exploit the structure of the drifting user. This leads to consistently lower average and cumulative rewards.

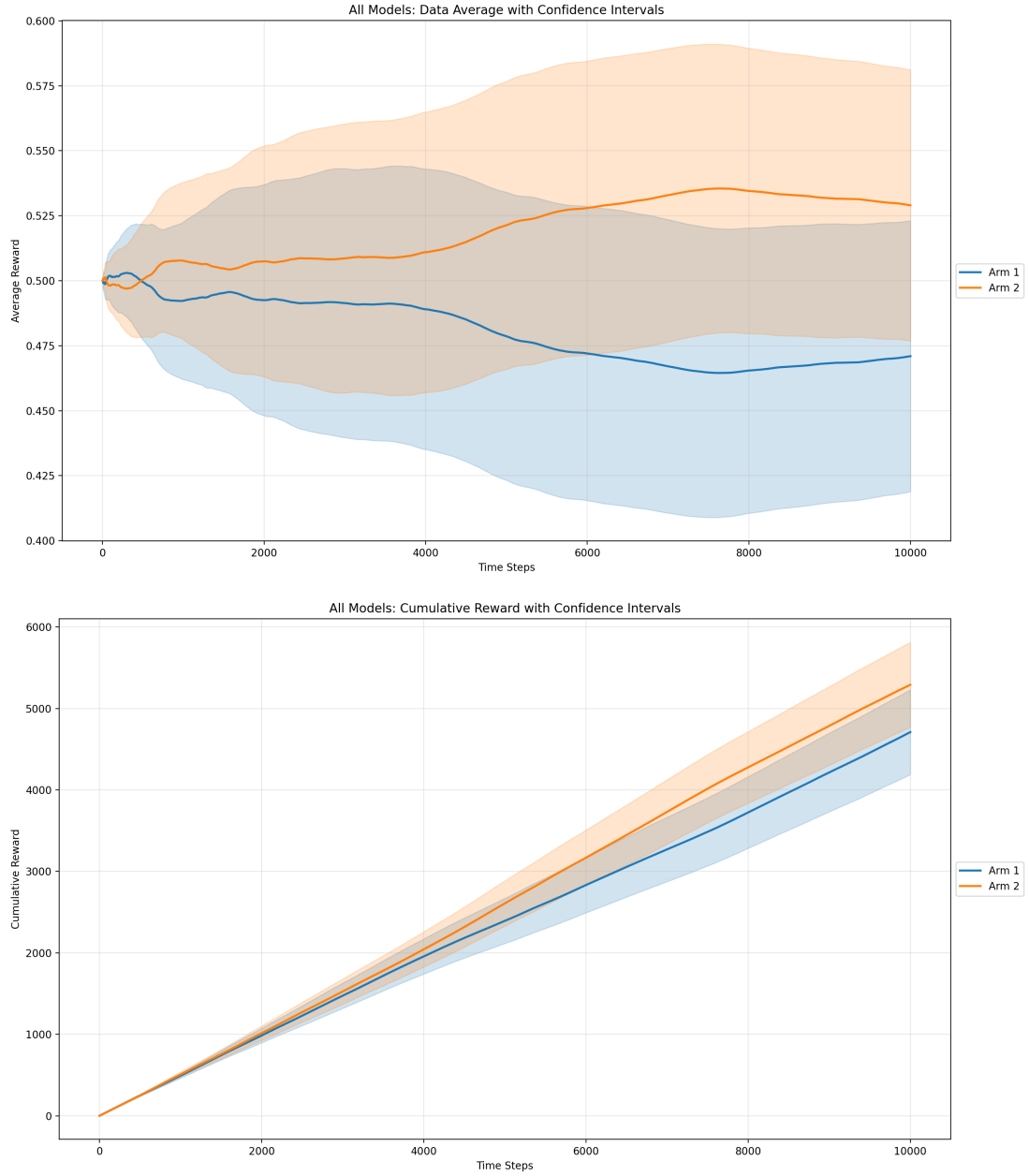


Figure 1: Drifting-user environment: rolling average (top) and cumulative reward (bottom) for each arm, averaged over 50 runs with 95% confidence intervals.

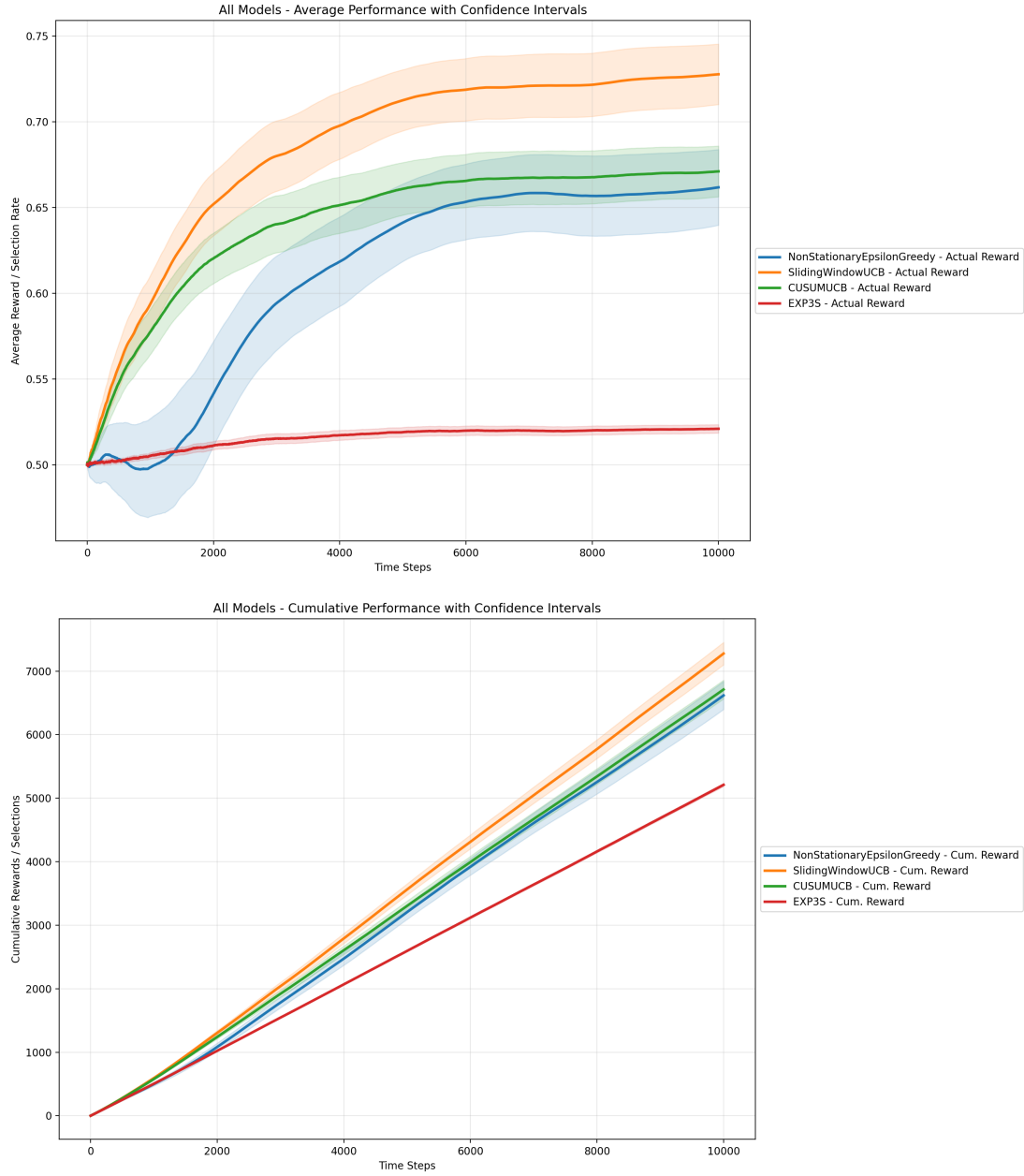


Figure 2: Mean rolling average reward (top) and mean cumulative reward (bottom) for all algorithms over 10,000 steps, averaged over 50 runs with 95% confidence intervals.

6 Discussion

The main challenge in this project was to disentangle algorithmic behavior from environmental effects. Synthetic drift parameters, horizon length, and hyperparameters all influence performance. By fixing a simple two-arm setting and running many seeds, we were able to uncover consistent patterns: methods that explicitly forget or reset old information adapt best to user drift.

Compared to related work, our implementation of SW-UCB and CUSUM-UCB confirms their advantage in non-stationary stochastic environments. Our experiments also highlight a common misconception: adversarial bandit algorithms such as EXP3.S, while theoretically appealing, are not necessarily competitive in smoothly drifting recommender scenarios where rewards are still generated from underlying distributions. For practitioners, this suggests prioritizing sliding-window or change-detection schemes before moving to adversarial methods.

7 Conclusion

This report presented a drifting-user simulator and a comparative study of four adaptive bandit algorithms. In our synthetic non-stationary environment, Sliding-Window UCB consistently achieved the highest long-run reward, followed by CUSUM-UCB and Non-Stationary ϵ -Greedy, with EXP3.S lagging behind. These results support the use of explicit forgetting mechanisms for recommender systems facing gradual preference drift.

Future extensions include richer context features, multiple users, more complex drift patterns, and offline evaluation on logged interaction data. A systematic hyperparameter study would also clarify how robust the observed ranking is across environments and design choices.

References

- [1] A. Garivier and E. Moulines, “On upper-confidence bound policies for non-stationary bandit problems,” 2008. [Online]. Available: <https://arxiv.org/abs/0805.3415>
- [2] R. S. Sutton, A. G. Barto *et al.*, “Reinforcement learning: An introduction,” vol. 1, no. 1, 1998.
- [3] F. Liu, J. Lee, and N. Shroff, “A change-detection based framework for piecewise-stationary multi-armed bandit problem,” 2017. [Online]. Available: <https://arxiv.org/abs/1711.03539>
- [4] N. Chen, S. Yang, and H. Zhang, “Bridging adversarial and nonstationary multi-armed bandit,” 2023. [Online]. Available: <https://arxiv.org/abs/2201.01628>
- [5] P. Auer, N. Cesa-Bianchi, Y. Freund, and R. E. Schapire, “The nonstochastic multiarmed bandit problem,” *SIAM J. Comput.*, vol. 32, pp. 48–77, 2002. [Online]. Available: <https://api.semanticscholar.org/CorpusID:13209702>